



호가창 스트림 저장 포맷의 설계 공간 분석

도메인 변환, 컬럼 레이아웃, 체크포인트 정책

발표자

김태우 zop1234@hanmail.net

순천향대학교 AI · 빅데이터학과 학사과정



목차

- 01 | 배경
- 02 | 문제 정의
- 03 | 연구 계획

호가창은 가격별 대기 주문의 합계표이고 현재가와는 다른 정보

- 호가창의 한 줄이 가격 레벨
- 숫자가 그 가격에 대기 중인 주문 합계
- 현재가는 마지막 체결가이기 때문에 표에 없음
- 호가창에는 가격의 유동성, 큰 주문의 위치같은 정보가 있어 미시구조 연구와 백테스트에 자주 쓰임

Order Book			0.01 ▾
Price (USDT)	Amount (BTC)	Total	
79,784.72	0.00014	11.169860	
79,784.04	0.00014	11.169765	
79,784.00	0.03028	2.41K	
79,783.99	0.00014	11.169758	
79,783.74	0.00062	49.465918	
79,783.27	0.05864	4.67K	
79,783.26	0.00014	11.169656	
79,782.14	0.00317	252.909383	
79,782.00	0.03028	2.41K	
79,781.99	0.03718	2.96K	
79,781.98	0.00007	5.584738	
79,781.94	0.00007	5.584735	
79,781.84	0.00727	580.013976	
79,781.61	0.01014	808.985525	
79,781.59	0.00013	10.371606	
79,781.58	0.00097	77.388132	
79,781.57	0.60249	48.06K	
79,781.57 ↑ \$79,781.57			>
79,781.56	2.56835	204.90K	
79,781.55	0.30027	23.95K	
79,781.54	0.00013	10.371600	
79,781.51	0.00014	11.169411	
79,781.50	0.30000	23.93K	
79,781.18	0.00014	11.169365	
79,781.17	1.07996	86.16K	
79,781.16	0.37500	29.91K	
79,781.15	0.01504	1.20K	
79,781.14	0.01025	818.554496	
79,780.08	0.06685	5.33K	
79,780.05	0.00007	5.584603	
79,780.00	0.00400	319.120000	
79,779.87	0.01710	1.36K	
79,779.77	0.07500	5.98K	

거래소는 변화분만 전송

호가 스트림은 표 전체가 아닌 바뀐 줄의 새 합계만 전송

예시 → (매수 50,000 → 2.5, 매수 49,999 → 0)

내용

- 주문 종류별로 합계 형태로 전송됨
- 0은 해당 가격의 레벨 삭제를 의미
- 주문 체결은 별도 스트림으로 전송됨
- 표 전체는 REST 스냅샷으로 한 번 받고 그 위에 변화분을 순번(U, u) 검증하며 적용해야 호가창이 유지됨
- 순번이 끊기면 갭이 발생하는 것이고 스냅샷을 다시 받아 동기화 진행 필요

호가 스트림 데이터의 규모와 성질

데이터의 양은 많지만 구조가 강함

1 데이터 규모

- 활발한 종목은 초당 수백~수천 이벤트 발생
- 종목 10개면 raw JSON으로 하루 수 GB 용량 필요

2 데이터 성질

- 가격과 수량은 호가 단위의 배수
- 시간은 거의 일정 간격으로 증가
- 변화는 최우선호가 근처에 집중
- 메시지에 키 이름이나 종목명 같은 반복이 많음

기존 저장 방식의 특징과 한계

1 NDJSON + zstd

- 받은 메시지를 한 줄씩 JSON 파일에 그대로 저장하고 압축하는 방법이며 원본 보존용이고 가장 단순함
- 장점: 원본이 그대로 남고 키 이름 같은 반복은 zstd를 통해 거의 압축됨
- 단점
 - 숫자를 10진 문자열로 저장해서 용량이 크고, 압축기는 “직전 값과 한 호가 단위 차이” 같은 숫자 구조를 모름
 - 일반 zstd 스트림은 중간부터 읽을 수 없어 특정 시각을 보려 해도 파일 앞부터 압축을 해제해야 함

2 Parquet

- 값을 열 단위로 모아서 저장하는 파일 형식이고, 정수 열에는 델타·사전·비트 패킹 인코딩을 자동으로 적용
- 장점: 집계와 구간 조회가 빠르고, 가격을 미리 호가 단위의 배수인 정수로 바꿔서 넣으면 작은 차이를 잘 압축함
- 단점
 - 가격을 정수로 바꾸려면 종목별 호가 단위를 알아야 하는데 Parquet은 이것을 모르기 때문에 직접 정수로 바꿔서 넣어야 함
 - 바뀐 가격이 최우선평가에서 몇 칸이 떨어졌는지, 이 변경이 레벨 추가인지 수정인지 삭제인지 알 수 없고 그만큼 압축 기회를 놓침

기존 저장 방식의 특징과 한계

3

DBN

- Databento가 Apache-2.0으로 공개한 시장 데이터 특화 바이너리 인코딩 포맷이고, 체결·호가 스키마가 고정돼 있고 zstd로 압축 가능
- 장점: 레코드가 고정 크기이기 때문에 용량이 작고 순서대로 읽기가 빠름
- 단점
 - 표준 스키마가 주문 단위(MBO)와 고정 레벨(MBP-1, MBP-10)뿐임
 - 바이낸스는 호가창에서 바뀐 가격 레벨만 보내고, 가격 레벨 수에 제한이 없음 → DBN은 이 데이터를 담을 스키마가 없음
 - 특정 시점 조회나 스프레드·mid-price 같은 요약값 계산은 DBN이 해주지 않아 직접 코드를 짜야 함

4

DuckDB · ClickHouse

- SQL로 조회하는 컬럼형 분석 DB이고, DuckDB는 프로그램에 내장하는 형태, ClickHouse는 서버로 띄우는 형태
- 장점: 집계와 구간 조회는 SQL 한 줄로 가능하고 ClickHouse는 열마다 압축 방식을 고를 수 있어 잘 맞추면 용량도 적음
- 단점: 특정 시점 복원은 스냅샷 테이블과 SQL을 직접 설계해야 하고, 호가창 구조는 모르고 결과가 설정에 크게 좌우됨

연구 질문 및 범위

연구 질문

Q1. 실시간으로 변화하는 호가창 데이터에는 압축에 쓸 만한 어떤 규칙성이 있는가

Q2. 호가 데이터에 맞춘 변환, 컬럼형 저장, 체크포인트 간격은 저장 용량과 조회 속도를 각각 얼마나 개선하는가

Q3. 이렇게 만든 저장 방식은 제대로 설정한 기존 도구와 비교해 얼마나 낫거나 못한가

연구 범위

- 바이낸스 현물 8~10종목
- 4종류 조회 - 특정 시점 호가창 복원, 하루치 순서대로 재생, 시간 범위 조회, 집계(스프레드·mid-price)

1 호가 데이터에 맞춘 변환

- 가격과 수량은 호가 단위의 배수 → 정수로 바꿔 저장
- 시각은 거의 일정한 간격으로 증가 → 앞 값과의 차이만 저장
- 변화는 최우선�호가 근처에 몰림 → 가격을 "최우선평가에서 몇 칸" 형태로 저장
- 변경은 가격 레벨 추가·수정·삭제 중 하나 → 무엇이 바뀌었는지와 얼마로 바뀌었는지를 따로 저장
- 키 이름과 종목명이 반복 될 때 → zstd 사전으로 압축

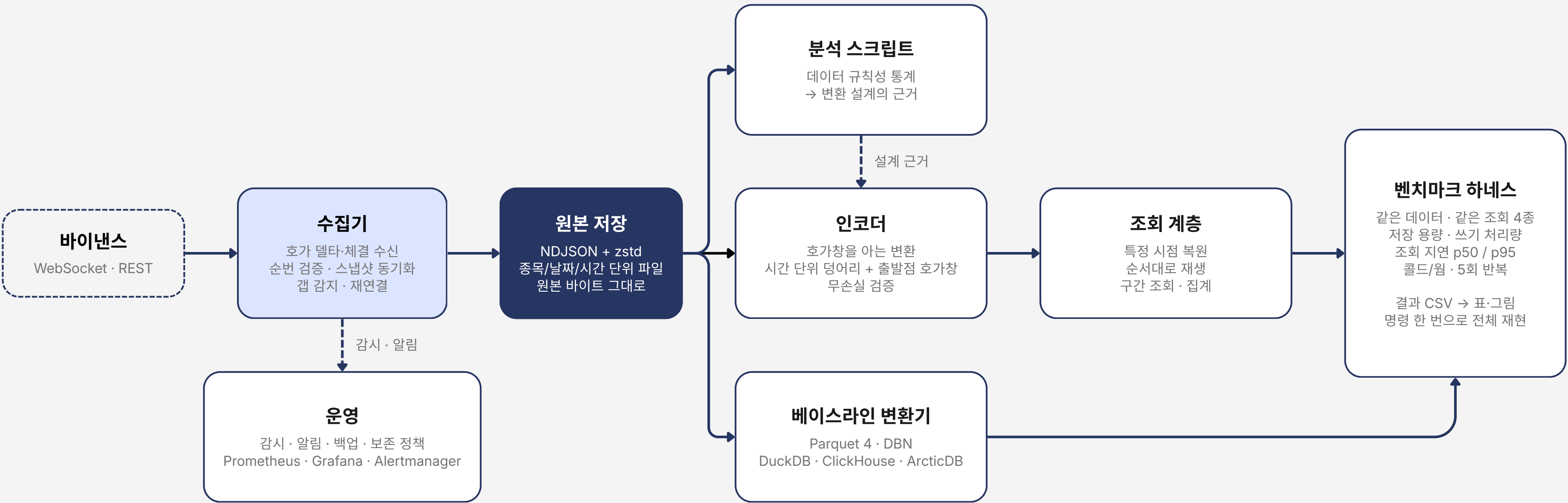
2 설계 요소별 효과 분석

- 1단계: Parquet에 그대로 저장
- 2단계: 정수로 바꿔 저장
- 3단계: 위의 호가 데이터에 맞춘 변환을 모두 적용해 저장
- 4단계: 3단계 변환에 특정 시점 조회용 구조(시간 단위 덩어리와 출발점 호가창)를 더한 것

03

연구 계획

시스템 구성





감사합니다

Q&A

발표자



김태우 zop1234@hanmail.net

순천향대학교 AI · 빅데이터학과 학사과정